Level 6: Window Functions

Window functions add a computed column alongside every row, based on a 'window' of related rows -- unlike GROUP BY, no rows get collapsed. This level covers ranking (ROW_NUMBER/RANK/DENSE_RANK), row comparisons (LEAD/LAG), running totals and moving averages, PARTITION BY, and the CTE-then-filter pattern needed for 'top N per group' queries.

Q141. ROW_NUMBER() - Rank employees by salary MEDIUM

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

WHAT THIS QUERY DOES

Welcome to **window functions** — a different beast from GROUP BY. GROUP BY *collapses* many rows into one summary row per group; a window function instead keeps *every original row* and just adds an extra computed column alongside it, based on a 'window' of related rows (here, the whole table, ordered by salary). `ROW_NUMBER()` simply numbers the rows 1, 2, 3... in the order specified by `OVER (ORDER BY salary DESC)` — the highest earner gets row_num 1, and every row gets a strictly unique number even if two salaries happen to tie.

RESULT ON OUR SAMPLE DATA

name	salary	row_num
Karthik Nair	95,000	1
Ravi Teja	91,000	2
Vikram Reddy	90,000	3
Suresh Babu	88,000	4
Rahul Singh	85,000	5
Arjun Reddy	82,000	6
Aarav Sharma	75,000	7
Neha Gupta	72,000	8
Rajesh Kumar	70,000	9
Meera Krishnan	68,000	10
Kavya Nair	67,000	11
Priya Patel	65,000	12
Deepa Iyer	64,000	13
Ananya Menon	60,000	14
Sneha Patel	58,000	15

15 rows returned.

Q142. RANK() - Rank with gaps MEDIUM

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

WHAT THIS QUERY DOES

`RANK()` is like `ROW_NUMBER()`, but it's aware of ties: two rows with the exact same salary get the exact same rank. The catch is that ranks then **skip** ahead — if two people tie for rank 2, the next distinct salary jumps straight to rank 4 (not 3), because 'rank 3' is considered already used up by the two-way tie. This mirrors how sports leaderboards usually work: two joint-2nd-place finishers mean there's no 3rd place at all.

RESULT ON OUR SAMPLE DATA

name	salary	salary_rank
Karthik Nair	95,000	1
Ravi Teja	91,000	2
Vikram Reddy	90,000	3
Suresh Babu	88,000	4
Rahul Singh	85,000	5
Arjun Reddy	82,000	6
Aarav Sharma	75,000	7
Neha Gupta	72,000	8
Rajesh Kumar	70,000	9
Meera Krishnan	68,000	10
Kavya Nair	67,000	11
Priya Patel	65,000	12
Deepa Iyer	64,000	13
Ananya Menon	60,000	14
Sneha Patel	58,000	15

15 rows returned.

Q143. DENSE_RANK() - Rank without gaps MEDIUM

```
SELECT name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

WHAT THIS QUERY DOES

`DENSE_RANK()` handles ties the same way as `RANK()` (equal values get equal ranks), but *without* leaving gaps afterward — after a two-way tie for rank 2, the next distinct value becomes rank 3, not rank 4. In our sample data every salary happens to be unique, so `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()` all produce identical numbers here — their differences only become visible once there are genuine ties, which is worth keeping in mind if a first look at the output makes them seem interchangeable.

RESULT ON OUR SAMPLE DATA

name	salary	dense_rank
Karthik Nair	95,000	1
Ravi Teja	91,000	2
Vikram Reddy	90,000	3
Suresh Babu	88,000	4
Rahul Singh	85,000	5
Arjun Reddy	82,000	6
Aarav Sharma	75,000	7
Neha Gupta	72,000	8
Rajesh Kumar	70,000	9
Meera Krishnan	68,000	10
Kavya Nair	67,000	11
Priya Patel	65,000	12
Deepa Iyer	64,000	13
Ananya Menon	60,000	14
Sneha Patel	58,000	15

15 rows returned.

Q144. LEAD() - Next salary MEDIUM

```
SELECT name, salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

WHAT THIS QUERY DOES

LEAD() reaches *forward* to grab a value from a later row (by default, the very next one) in the specified order, without needing a self-join. Ordered ascending here, each row's next_salary shows what the *next-higher-paid* employee earns — the very last row (the highest earner) has nothing after it, so its next_salary comes back NULL.

RESULT ON OUR SAMPLE DATA

name	salary	next_salary
Sneha Patel	58,000	60,000
Ananya Menon	60,000	64,000
Deepa Iyer	64,000	65,000
Priya Patel	65,000	67,000
Kavya Nair	67,000	68,000
Meera Krishnan	68,000	70,000
Rajesh Kumar	70,000	72,000
Neha Gupta	72,000	75,000
Aarav Sharma	75,000	82,000
Arjun Reddy	82,000	85,000
Rahul Singh	85,000	88,000
Suresh Babu	88,000	90,000
Vikram Reddy	90,000	91,000
Ravi Teja	91,000	95,000
Karthik Nair	95,000	*NULL*

15 rows returned.

Q145. LAG() - Previous salary MEDIUM

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary
FROM employees;
```

WHAT THIS QUERY DOES

The mirror image of LEAD() — LAG() reaches *backward* to the previous row instead. The very first row (lowest salary) has nothing before it, so its prev_salary is NULL. Together, LEAD and LAG are what let you compare each row to its neighbours without writing a self-join or a correlated subquery.

RESULT ON OUR SAMPLE DATA

name	salary	prev_salary
Sneha Patel	58,000	*NULL*

name	salary	prev_salary
Ananya Menon	60,000	58,000
Deepa Iyer	64,000	60,000
Priya Patel	65,000	64,000
Kavya Nair	67,000	65,000
Meera Krishnan	68,000	67,000
Rajesh Kumar	70,000	68,000
Neha Gupta	72,000	70,000
Aarav Sharma	75,000	72,000
Arjun Reddy	82,000	75,000
Rahul Singh	85,000	82,000
Suresh Babu	88,000	85,000
Vikram Reddy	90,000	88,000
Ravi Teja	91,000	90,000
Karthik Nair	95,000	91,000

15 rows returned.

Q146. FIRST_VALUE() - Lowest salary in department HARD

```
SELECT name, department, salary,
       FIRST_VALUE(name) OVER (PARTITION BY department ORDER BY salary) AS
lowest_paid
FROM employees;
```

WHAT THIS QUERY DOES

`PARTITION BY` is to window functions what `GROUP BY` is to aggregates — it splits the table into per-department 'windows,' and everything in `OVER(...)` then operates *separately within each partition* instead of across the whole table. `FIRST_VALUE()` grabs the value from the very first row of each partition (as defined by its `ORDER BY`) — here, ascending salary order means 'first' means 'lowest-paid,' so every row in a department shows that department's lowest earner's name, repeated alongside each of that department's individual rows.

RESULT ON OUR SAMPLE DATA

name	department	salary	lowest_paid
Kavya Nair	Finance	67,000	Kavya Nair
Rajesh Kumar	Finance	70,000	Kavya Nair
Rahul Singh	Finance	85,000	Kavya Nair
Ananya Menon	HR	60,000	Ananya Menon
Deepa Iyer	HR	64,000	Ananya Menon

name	department	salary	lowest_paid
Priya Patel	HR	65,000	Ananya Menon
Neha Gupta	IT	72,000	Neha Gupta
Aarav Sharma	IT	75,000	Neha Gupta
Arjun Reddy	IT	82,000	Neha Gupta
Suresh Babu	IT	88,000	Neha Gupta
Vikram Reddy	IT	90,000	Neha Gupta
Ravi Teja	IT	91,000	Neha Gupta
Karthik Nair	IT	95,000	Neha Gupta
Sneha Patel	Marketing	58,000	Sneha Patel
Meera Krishnan	Marketing	68,000	Sneha Patel

15 rows returned.

Q147. LAST_VALUE() - Highest salary in department HARD

```
SELECT name, department, salary,
       LAST_VALUE(name) OVER (PARTITION BY department ORDER BY salary
                              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING) AS highest_paid
FROM employees;
```

WHAT THIS QUERY DOES

The counterpart to Q146, but with an important gotcha: by default, a window function's 'frame' only extends up to the *current row* (not the whole partition), so a naive `LAST_VALUE()` would just return each row's *own* name, not the true partition maximum. The extra clause `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` explicitly widens the frame to the *entire* partition, every time, regardless of which row is 'current' — only then does `LAST_VALUE()` correctly return the highest-paid person's name for every row in that department.

RESULT ON OUR SAMPLE DATA

name	department	salary	highest_paid
Kavya Nair	Finance	67,000	Rahul Singh
Rajesh Kumar	Finance	70,000	Rahul Singh
Rahul Singh	Finance	85,000	Rahul Singh
Ananya Menon	HR	60,000	Priya Patel
Deepa Iyer	HR	64,000	Priya Patel
Priya Patel	HR	65,000	Priya Patel
Neha Gupta	IT	72,000	Karthik Nair
Aarav Sharma	IT	75,000	Karthik Nair

name	department	salary	highest_paid
Arjun Reddy	IT	82,000	Karthik Nair
Suresh Babu	IT	88,000	Karthik Nair
Vikram Reddy	IT	90,000	Karthik Nair
Ravi Teja	IT	91,000	Karthik Nair
Karthik Nair	IT	95,000	Karthik Nair
Sneha Patel	Marketing	58,000	Meera Krishnan
Meera Krishnan	Marketing	68,000	Meera Krishnan

15 rows returned.

Q148. NTILE() - Divide into quartiles MEDIUM

```
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary) AS salary_quartile
FROM employees;
```

WHAT THIS QUERY DOES

`NTILE(4)` divides the ordered rows into 4 roughly-equal-sized buckets (quartiles) and labels each row with which bucket it landed in — bucket 1 holds the lowest-salary employees, bucket 4 the highest. With 15 employees split 4 ways, some buckets naturally end up with 4 people and others with 3, since 15 doesn't divide evenly by 4; `NTILE()` handles that automatically by giving the earlier buckets the extra rows.

RESULT ON OUR SAMPLE DATA

name	salary	salary_quartile
Sneha Patel	58,000	1
Ananya Menon	60,000	1
Deepa Iyer	64,000	1
Priya Patel	65,000	1
Kavya Nair	67,000	2
Meera Krishnan	68,000	2
Rajesh Kumar	70,000	2
Neha Gupta	72,000	2
Aarav Sharma	75,000	3
Arjun Reddy	82,000	3
Rahul Singh	85,000	3
Suresh Babu	88,000	3
Vikram Reddy	90,000	4
Ravi Teja	91,000	4

name	salary	salary_quartile
Karthik Nair	95,000	4

15 rows returned.

Q149. Running total of salaries MEDIUM

```
SELECT name, salary,
       SUM(salary) OVER (ORDER BY emp_id) AS running_total
FROM employees;
```

WHAT THIS QUERY DOES

A **running total**: ordering by `emp_id`, each row's `running_total` is the sum of every salary *up to and including* the current row. This works because, as mentioned in Q147, a window function's default frame is 'from the start of the partition up through the current row' when an `ORDER BY` is present — which is exactly the shape a running total needs, with no extra `ROWS BETWEEN` clause required.

RESULT ON OUR SAMPLE DATA

name	salary	running_total
Aarav Sharma	75,000	75,000
Priya Patel	65,000	140,000
Rahul Singh	85,000	225,000
Neha Gupta	72,000	297,000
Vikram Reddy	90,000	387,000
Ananya Menon	60,000	447,000
Karthik Nair	95,000	542,000
Meera Krishnan	68,000	610,000
Arjun Reddy	82,000	692,000
Sneha Patel	58,000	750,000
Rajesh Kumar	70,000	820,000
Deepa Iyer	64,000	884,000
Suresh Babu	88,000	972,000
Kavya Nair	67,000	1,039,000
Ravi Teja	91,000	1,130,000

15 rows returned.

Q150. Cumulative salary by department MEDIUM

```
SELECT name, department, salary,
       SUM(salary) OVER (PARTITION BY department ORDER BY emp_id) AS
dept_cumulative
FROM employees;
```

WHAT THIS QUERY DOES

Combines the running-total idea from Q149 with PARTITION BY from Q146 — now the running total resets separately at the start of each department, rather than accumulating across the whole table. This is the standard shape for 'running total, but broken out per category' reports.

RESULT ON OUR SAMPLE DATA

name	department	salary	dept_cumulative
Rahul Singh	Finance	85,000	85,000
Rajesh Kumar	Finance	70,000	155,000
Kavya Nair	Finance	67,000	222,000
Priya Patel	HR	65,000	65,000
Ananya Menon	HR	60,000	125,000
Deepa Iyer	HR	64,000	189,000
Aarav Sharma	IT	75,000	75,000
Neha Gupta	IT	72,000	147,000
Vikram Reddy	IT	90,000	237,000
Karthik Nair	IT	95,000	332,000
Arjun Reddy	IT	82,000	414,000
Suresh Babu	IT	88,000	502,000
Ravi Teja	IT	91,000	593,000
Meera Krishnan	Marketing	68,000	68,000
Sneha Patel	Marketing	58,000	126,000

15 rows returned.

Q151. Moving average of last 3 employees HARD

```
SELECT name, salary,
       AVG(salary) OVER (ORDER BY emp_id ROWS BETWEEN 2 PRECEDING AND CURRENT
ROW) AS moving_avg
FROM employees;
```

WHAT THIS QUERY DOES

A **moving average**: instead of the default 'from the start' frame, `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` explicitly defines a sliding 3-row window (the current row plus the 2 before it), and `AVG()` is computed fresh within that window at every position. Near the very start of the table, where fewer than 2 prior rows exist, the window simply shrinks to whatever's available rather than erroring out.

RESULT ON OUR SAMPLE DATA

name	salary	moving_avg
Aarav Sharma	75,000	75,000
Priya Patel	65,000	70,000
Rahul Singh	85,000	75,000
Neha Gupta	72,000	74,000
Vikram Reddy	90,000	82,333.33
Ananya Menon	60,000	74,000
Karthik Nair	95,000	81,666.67
Meera Krishnan	68,000	74,333.33
Arjun Reddy	82,000	81,666.67
Sneha Patel	58,000	69,333.33
Rajesh Kumar	70,000	70,000
Deepa Iyer	64,000	64,000
Suresh Babu	88,000	74,000
Kavya Nair	67,000	73,000
Ravi Teja	91,000	82,000

15 rows returned.

Q152. Top 3 salaries per department HARD

```
WITH ranked AS (
    SELECT name, department, salary,
           DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dr
    FROM employees
)
SELECT department, name, salary
FROM ranked
WHERE dr <= 3;
```

WHAT THIS QUERY DOES

This introduces a **CTE** (Common Table Expression, the `WITH ... AS (...)` syntax) — a named, temporary result set you can reference like a table in the query that follows. It's needed here because of a strict SQL rule: you're **not allowed to filter directly on a window function's result in the same SELECT that computes it** (writing `WHERE dr <= 3` right next to `DENSE_RANK() ... AS dr` would be a syntax

error). The workaround is always the same two-step shape: compute the ranking inside a CTE first, then filter on it in an outer query that reads from the CTE. This exact pattern — rank inside a CTE, filter outside — is one of the most useful things to memorize from this whole book.

RESULT ON OUR SAMPLE DATA

department	name	salary
Finance	Rahul Singh	85,000
Finance	Rajesh Kumar	70,000
Finance	Kavya Nair	67,000
HR	Priya Patel	65,000
HR	Deepa Iyer	64,000
HR	Ananya Menon	60,000
IT	Karthik Nair	95,000
IT	Ravi Teja	91,000
IT	Vikram Reddy	90,000
Marketing	Meera Krishnan	68,000
Marketing	Sneha Patel	58,000

11 rows returned.

Q153. Lowest salary per department HARD

```
WITH ranked AS (
    SELECT name, department, salary,
           RANK() OVER (PARTITION BY department ORDER BY salary) AS rn
    FROM employees
)
SELECT department, name, salary
FROM ranked
WHERE rn = 1;
```

WHAT THIS QUERY DOES

The same CTE-then-filter shape as Q152, just ranking ascending (so rank 1 means *lowest* salary) and keeping only rank 1. This is functionally the same result as the correlated-subquery approach in Q128 — a good example of two completely different techniques (window function + CTE vs. correlated subquery) arriving at the same answer, each with its own strengths (window functions tend to be easier to extend to 'top N' as done in Q152, while correlated subqueries can be simpler for a single MIN/MAX).

RESULT ON OUR SAMPLE DATA

department	name	salary
Finance	Kavya Nair	67,000
HR	Ananya Menon	60,000
IT	Neha Gupta	72,000
Marketing	Sneha Patel	58,000

4 rows returned.

Q154. Previous salary comparison MEDIUM

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary,
       salary - LAG(salary) OVER (ORDER BY salary) AS diff_from_prev
FROM employees;
```

WHAT THIS QUERY DOES

Builds directly on Q145 by using the same LAG() value twice — once to display it, and once inside a subtraction to compute the gap between each employee's salary and the person just below them in the ranking. The very first row's difference is NULL, since LAG() itself is NULL there and any arithmetic involving NULL also produces NULL.

RESULT ON OUR SAMPLE DATA

name	salary	prev_salary	diff_from_prev
Sneha Patel	58,000	*NULL*	*NULL*
Ananya Menon	60,000	58,000	2,000
Deepa Iyer	64,000	60,000	4,000
Priya Patel	65,000	64,000	1,000
Kavya Nair	67,000	65,000	2,000
Meera Krishnan	68,000	67,000	1,000
Rajesh Kumar	70,000	68,000	2,000
Neha Gupta	72,000	70,000	2,000
Aarav Sharma	75,000	72,000	3,000
Arjun Reddy	82,000	75,000	7,000
Rahul Singh	85,000	82,000	3,000
Suresh Babu	88,000	85,000	3,000
Vikram Reddy	90,000	88,000	2,000
Ravi Teja	91,000	90,000	1,000
Karthik Nair	95,000	91,000	4,000

15 rows returned.

Q155. Next salary comparison MEDIUM

```
SELECT name, salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary,
       LEAD(salary) OVER (ORDER BY salary) - salary AS diff_to_next
FROM employees;
```

WHAT THIS QUERY DOES

The LEAD() counterpart to Q154 — shows the gap up to the *next*-higher salary instead of down to the previous one. Comparing the two diff columns across both queries is a good way to see that 'gap to the next person up' and 'gap from the previous person' are just the same set of gaps, viewed from opposite directions.

RESULT ON OUR SAMPLE DATA

name	salary	next_salary	diff_to_next
Sneha Patel	58,000	60,000	2,000
Ananya Menon	60,000	64,000	4,000
Deepa Iyer	64,000	65,000	1,000
Priya Patel	65,000	67,000	2,000
Kavya Nair	67,000	68,000	1,000
Meera Krishnan	68,000	70,000	2,000
Rajesh Kumar	70,000	72,000	2,000
Neha Gupta	72,000	75,000	3,000
Aarav Sharma	75,000	82,000	7,000
Arjun Reddy	82,000	85,000	3,000
Rahul Singh	85,000	88,000	3,000
Suresh Babu	88,000	90,000	2,000
Vikram Reddy	90,000	91,000	1,000
Ravi Teja	91,000	95,000	4,000
Karthik Nair	95,000	*NULL*	*NULL*

15 rows returned.

Q156. Month-over-month sales HARD

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(amount) AS total_sales,
       LAG(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) AS
prev_month_sales,
       (SUM(amount) - LAG(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date,
'%Y-%m'))) /
        LAG(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) * 100
```

```
AS growth_percent
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m');
```

WHAT THIS QUERY DOES

This shows that `LAG()/LEAD()` can be applied *on top of an aggregate*, not just on raw columns: the query first groups orders by month and sums each month's revenue, and only then does `LAG(SUM(amount))` reach back to grab the *previous month's already-summed total* — letting you compute a month-over-month growth percentage in a single query, without a self-join or a separate 'previous month' subquery. *(`DATE_FORMAT()` is a MySQL function; the live example uses SQLite's `strftime('%Y-%m', order_date)` to the same effect.)*

RESULT ON OUR SAMPLE DATA

month	total_sales	prev_month_sales	growth_percent
2024-01	50,000	*NULL*	*NULL*
2024-02	145,000	50,000	190
2024-03	150,000	145,000	3.45
2024-04	119,000	150,000	-20.67

4 rows returned.

Q157. Daily revenue with running total MEDIUM

```
SELECT order_date,
       SUM(amount) AS daily_revenue,
       SUM(SUM(amount)) OVER (ORDER BY order_date) AS cumulative_revenue
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

WHAT THIS QUERY DOES

Notice the **nested SUMs**: the inner `SUM(amount)` is a normal GROUP BY aggregate producing one total per day, while the outer `SUM(...) OVER (ORDER BY order_date)` is a window function running a cumulative total *across those already-aggregated daily totals*. Aggregate functions and window functions can combine like this — it's a slightly advanced-looking pattern, but it's really just 'Q149's running total, applied to grouped daily sums instead of individual raw rows.'

RESULT ON OUR SAMPLE DATA

order_date	daily_revenue	cumulative_revenue
2024-01-10	15,000	15,000
2024-01-15	23,000	38,000
2024-01-20	12,000	50,000

order_date	daily_revenue	cumulative_revenue
2024-02-01	50,000	100,000
2024-02-05	18,000	118,000
2024-02-10	32,000	150,000
2024-02-15	45,000	195,000
2024-03-01	21,000	216,000
2024-03-05	67,000	283,000
2024-03-10	34,000	317,000
2024-03-15	28,000	345,000
2024-04-01	42,000	387,000
2024-04-05	15,000	402,000
2024-04-10	33,000	435,000
2024-04-15	29,000	464,000

15 rows returned.

Q158. Customer ranking by total spent MEDIUM

```
SELECT c.name,
       SUM(o.amount) AS total_spent,
       RANK() OVER (ORDER BY SUM(o.amount) DESC) AS customer_rank
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

WHAT THIS QUERY DOES

JOIN + GROUP BY (to compute each customer's total spend) combined with a window function (to rank those totals) — all three major query-building techniques from this book working together in one query. This is a very realistic 'top customers' report shape.

RESULT ON OUR SAMPLE DATA

name	total_spent	customer_rank
InnovationLabs	97,000	1
DeltaCorp	67,000	2
TechCorp	53,000	3
GlobalSolutions	45,000	4
BetaSystems	45,000	4
OmegaLabs	42,000	6
EpsilonSoft	34,000	7
AlphaInc	32,000	8

name	total_spent	customer_rank
ThetaSolutions	28,000	9
GammaTech	21,000	10

10 rows returned.

Q159. Product ranking by sales MEDIUM

```
SELECT p.product_name,
       SUM(oi.quantity) AS total_quantity_sold,
       RANK() OVER (ORDER BY SUM(oi.quantity) DESC) AS product_rank
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name;
```

WHAT THIS QUERY DOES

The same JOIN + GROUP BY + RANK shape as Q158, applied to product sales volume instead of customer spend. LEFT JOIN (rather than a plain JOIN) ensures a product that's never been ordered still shows up ranked last, with a quantity of 0, instead of being silently excluded — the same reasoning as Q104.

RESULT ON OUR SAMPLE DATA

product_name	total_quantity_sold	product_rank
Laptop Pro	9	1
External SSD	4	2
27" Monitor	3	3
Enterprise Software	3	3
Wireless Mouse	1	5
Mechanical Keyboard	1	5
Cloud Storage Plan	1	5
Webcam HD	*NULL*	8
Printer	*NULL*	8
USB Hub	*NULL*	8

10 rows returned.

Q160. Department ranking by total salary EASY

```
SELECT department,
       SUM(salary) AS total_salary,
       RANK() OVER (ORDER BY SUM(salary) DESC) AS dept_rank
```

```
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Same GROUP BY + RANK combination once more, this time needing no JOIN at all since department and salary already live in the same table — a simpler variant of the Q158/159 shape.

RESULT ON OUR SAMPLE DATA

department	total_salary	dept_rank
IT	593,000	1
Finance	222,000	2
HR	189,000	3
Marketing	126,000	4

4 rows returned.

Q161. Percentage of total salary by employee MEDIUM

```
SELECT name, salary,
       ROUND(salary * 100.0 / SUM(salary) OVER (), 2) AS pct_of_total
FROM employees
ORDER BY pct_of_total DESC;
```

WHAT THIS QUERY DOES

An empty `OVER ()` — with no `PARTITION BY` and no `ORDER BY` — means 'treat the entire table as one single window.' `SUM(salary) OVER ()` therefore computes the grand total salary *once*, and repeats that same total alongside every row, which is exactly what you need as the denominator to calculate what percentage of total payroll each individual employee represents.

RESULT ON OUR SAMPLE DATA

name	salary	pct_of_total
Karthik Nair	95,000	8.41
Ravi Teja	91,000	8.05
Vikram Reddy	90,000	7.96
Suresh Babu	88,000	7.79
Rahul Singh	85,000	7.52
Arjun Reddy	82,000	7.26
Aarav Sharma	75,000	6.64
Neha Gupta	72,000	6.37
Rajesh Kumar	70,000	6.19

name	salary	pct_of_total
Meera Krishnan	68,000	6.02
Kavya Nair	67,000	5.93
Priya Patel	65,000	5.75
Deepa Iyer	64,000	5.66
Ananya Menon	60,000	5.31
Sneha Patel	58,000	5.13

15 rows returned.

Q162. Percentage within department MEDIUM

```
SELECT name, department, salary,
       ROUND(salary * 100.0 / SUM(salary) OVER (PARTITION BY department), 2) AS
pct_of_dept
FROM employees
ORDER BY department, pct_of_dept DESC;
```

WHAT THIS QUERY DOES

The same percentage-of-total idea as Q161, but adding `PARTITION BY department` so the denominator becomes each department's own total instead of the company-wide total — answering 'what share of my *department*'s payroll do I represent' rather than 'what share of the whole company.'

RESULT ON OUR SAMPLE DATA

name	department	salary	pct_of_dept
Rahul Singh	Finance	85,000	38.29
Rajesh Kumar	Finance	70,000	31.53
Kavya Nair	Finance	67,000	30.18
Priya Patel	HR	65,000	34.39
Deepa Iyer	HR	64,000	33.86
Ananya Menon	HR	60,000	31.75
Karthik Nair	IT	95,000	16.02
Ravi Teja	IT	91,000	15.35
Vikram Reddy	IT	90,000	15.18
Suresh Babu	IT	88,000	14.84
Arjun Reddy	IT	82,000	13.83
Aarav Sharma	IT	75,000	12.65
Neha Gupta	IT	72,000	12.14
Meera Krishnan	Marketing	68,000	53.97
Sneha Patel	Marketing	58,000	46.03

15 rows returned.

Q163. Salary percentile rank HARD

```
SELECT name, salary,
       PERCENT_RANK() OVER (ORDER BY salary) AS percentile_rank
FROM employees;
```

WHAT THIS QUERY DOES

PERCENT_RANK() expresses each row's rank as a fraction between 0 and 1, using the formula (rank - 1) / (total rows - 1) — the lowest value always scores exactly 0, the highest always scores exactly 1, and everyone else falls proportionally in between. This is the standard way to answer 'what percentile is this salary in' without manually computing rank-to-percentage conversions yourself.

RESULT ON OUR SAMPLE DATA

name	salary	percentile_rank
Sneha Patel	58,000	0
Ananya Menon	60,000	0.07
Deepa Iyer	64,000	0.14
Priya Patel	65,000	0.21
Kavya Nair	67,000	0.29
Meera Krishnan	68,000	0.36
Rajesh Kumar	70,000	0.43
Neha Gupta	72,000	0.50
Aarav Sharma	75,000	0.57
Arjun Reddy	82,000	0.64
Rahul Singh	85,000	0.71
Suresh Babu	88,000	0.79
Vikram Reddy	90,000	0.86
Ravi Teja	91,000	0.93
Karthik Nair	95,000	1

15 rows returned.

Q164. Cumulative distribution of salaries HARD

```
SELECT name, salary,
       CUME_DIST() OVER (ORDER BY salary) AS cume_distribution
FROM employees;
```

WHAT THIS QUERY DOES

Closely related to `PERCENT_RANK()` but subtly different: `CUME_DIST()` (cumulative distribution) reports, for each row, the fraction of *all* rows whose value is less than or equal to the current one. Unlike `PERCENT_RANK()`, the lowest value here doesn't score 0 — it scores 1/15 (itself counts as 'less than or equal to itself'), and the highest value always scores exactly 1.

RESULT ON OUR SAMPLE DATA

name	salary	cume_distribution
Sneha Patel	58,000	0.07
Ananya Menon	60,000	0.13
Deepa Iyer	64,000	0.20
Priya Patel	65,000	0.27
Kavya Nair	67,000	0.33
Meera Krishnan	68,000	0.40
Rajesh Kumar	70,000	0.47
Neha Gupta	72,000	0.53
Aarav Sharma	75,000	0.60
Arjun Reddy	82,000	0.67
Rahul Singh	85,000	0.73
Suresh Babu	88,000	0.80
Vikram Reddy	90,000	0.87
Ravi Teja	91,000	0.93
Karthik Nair	95,000	1

15 rows returned.

Q165. Employee tenure ranking MEDIUM

```
SELECT name,
       DATEDIFF(CURDATE(), hire_date) AS tenure_days,
       ROW_NUMBER() OVER (ORDER BY DATEDIFF(CURDATE(), hire_date) DESC) AS
tenure_rank
FROM employees;
```

WHAT THIS QUERY DOES

`DATEDIFF()` computes the number of days between two dates, and `CURDATE()` returns today's date — together giving each employee's tenure in days, which `ROW_NUMBER()` then ranks from longest-serving to newest. *Since 'today' would make this book's printed numbers go stale the moment you read it, the live example below fixes the reference date at 2024-12-31 (using SQLite's `julianday()` date arithmetic) instead of the real current date, so the tenure figures shown stay consistent and reproducible.*

RESULT ON OUR SAMPLE DATA

name	tenure_days	tenure_rank
Suresh Babu	3,958	1
Karthik Nair	3,380	2
Ravi Teja	3,091	3
Arjun Reddy	2,952	4
Vikram Reddy	2,593	5
Rahul Singh	2,366	6
Priya Patel	2,113	7
Rajesh Kumar	2,031	8
Aarav Sharma	1,812	9
Meera Krishnan	1,602	10
Deepa Iyer	1,507	11
Neha Gupta	1,429	12
Kavya Nair	1,229	13
Ananya Menon	930	14
Sneha Patel	711	15

15 rows returned.

Q166. Moving 7-day average of orders HARD

```
WITH daily_orders AS (
    SELECT order_date, COUNT(*) AS order_count
    FROM orders
    GROUP BY order_date
)
SELECT order_date, order_count,
       AVG(order_count) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS moving_7day_avg
FROM daily_orders
ORDER BY order_date;
```

WHAT THIS QUERY DOES

Combines a CTE (first collapse orders down to one row per day with its order count) and a sliding-window moving average (same `ROWS BETWEEN ... PRECEDING AND CURRENT ROW` idea as Q151, just widened from 3 rows to 7). Since our sample data only has one order per distinct date, the '7-day' window here mostly just averages however many daily-order-count rows came before it — but on denser real-world data, this is exactly how you'd smooth out day-to-day noise in an orders-per-day chart.

RESULT ON OUR SAMPLE DATA

order_date	order_count	moving_7day_avg
2024-01-10	1	1
2024-01-15	1	1
2024-01-20	1	1
2024-02-01	1	1
2024-02-05	1	1
2024-02-10	1	1
2024-02-15	1	1
2024-03-01	1	1
2024-03-05	1	1
2024-03-10	1	1
2024-03-15	1	1
2024-04-01	1	1
2024-04-05	1	1
2024-04-10	1	1
2024-04-15	1	1

15 rows returned.

Q167. Salary difference from department average MEDIUM

```
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY department) AS diff_from_dept_avg
FROM employees;
```

WHAT THIS QUERY DOES

A window-function alternative to the correlated subquery from Q123/116: `AVG(salary) OVER (PARTITION BY department)` computes each department's average salary and attaches it directly alongside every one of that department's rows, letting you compute the difference in the very same SELECT — generally considered more readable (and often faster) than the equivalent correlated subquery.

RESULT ON OUR SAMPLE DATA

name	department	salary	dept_avg	diff_from_dept_avg
Rahul Singh	Finance	85,000	74,000	11,000
Rajesh Kumar	Finance	70,000	74,000	-4,000
Kavya Nair	Finance	67,000	74,000	-7,000
Priya Patel	HR	65,000	63,000	2,000
Ananya Menon	HR	60,000	63,000	-3,000
Deepa Iyer	HR	64,000	63,000	1,000
Aarav Sharma	IT	75,000	84,714.29	-9,714.29
Neha Gupta	IT	72,000	84,714.29	-12,714.29
Vikram Reddy	IT	90,000	84,714.29	5,285.71
Karthik Nair	IT	95,000	84,714.29	10,285.71
Arjun Reddy	IT	82,000	84,714.29	-2,714.29
Suresh Babu	IT	88,000	84,714.29	3,285.71
Ravi Teja	IT	91,000	84,714.29	6,285.71
Meera Krishnan	Marketing	68,000	63,000	5,000
Sneha Patel	Marketing	58,000	63,000	-5,000

15 rows returned.

Q168. Running total per month MEDIUM

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(amount) AS monthly_sales,
       SUM(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) AS
cumulative_ytd
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m');
```

WHAT THIS QUERY DOES

The same 'nested SUM' running-total idea from Q157, now grouped by month instead of by day — producing a year-to-date cumulative revenue figure that grows month over month. *(Again using SQLite's `strftime('%Y-%m', ...)` in place of MySQL's `DATE_FORMAT()`.)*

RESULT ON OUR SAMPLE DATA

month	monthly_sales	cumulative_ytd
2024-01	50,000	50,000
2024-02	145,000	195,000
2024-03	150,000	345,000
2024-04	119,000	464,000

4 rows returned.

Q169. First order for each customer MEDIUM

```
SELECT customer_id, order_date, amount,
       FIRST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date) AS first_order_date
FROM orders;
```

WHAT THIS QUERY DOES

The window-function alternative to Q129/130's correlated-subquery approach: `FIRST_VALUE()` with `PARTITION BY customer_id` attaches each customer's earliest order date to every one of their order rows directly, without needing a separate subquery per row.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount	first_order_date
1	2024-01-10	15,000	2024-01-10
1	2024-01-15	23,000	2024-01-10
1	2024-04-05	15,000	2024-01-10
2	2024-01-20	12,000	2024-01-20
2	2024-04-10	33,000	2024-01-20
3	2024-02-01	50,000	2024-02-01
3	2024-02-05	18,000	2024-02-01
3	2024-04-15	29,000	2024-02-01
4	2024-02-10	32,000	2024-02-10
5	2024-02-15	45,000	2024-02-15
6	2024-03-01	21,000	2024-03-01
7	2024-03-05	67,000	2024-03-05
8	2024-03-10	34,000	2024-03-10
9	2024-03-15	28,000	2024-03-15
10	2024-04-01	42,000	2024-04-01

15 rows returned.

Q170. Last order for each customer MEDIUM

```
SELECT customer_id, order_date, amount,
       LAST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY order_date
                                    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
```

```
FOLLOWING) AS last_order_date
FROM orders;
```

WHAT THIS QUERY DOES

The `LAST_VALUE()` counterpart to Q169 — and just like Q147, it needs the explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` frame to correctly look at the *entire* partition rather than defaulting to 'up through the current row,' which would otherwise make every row just echo its own date back.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount	last_order_date
1	2024-01-10	15,000	2024-04-05
1	2024-01-15	23,000	2024-04-05
1	2024-04-05	15,000	2024-04-05
2	2024-01-20	12,000	2024-04-10
2	2024-04-10	33,000	2024-04-10
3	2024-02-01	50,000	2024-04-15
3	2024-02-05	18,000	2024-04-15
3	2024-04-15	29,000	2024-04-15
4	2024-02-10	32,000	2024-02-10
5	2024-02-15	45,000	2024-02-15
6	2024-03-01	21,000	2024-03-01
7	2024-03-05	67,000	2024-03-05
8	2024-03-10	34,000	2024-03-10
9	2024-03-15	28,000	2024-03-15
10	2024-04-01	42,000	2024-04-01

15 rows returned.